

Computação Gráfica em Tempo Real

prof. Guilherme Chagas Kurtz

Computação Gráfica em Tempo Real

Computação gráfica em tempo real trata da **geração contínua de imagens** a partir de dados gráficos, com resposta imediata às interações do usuário.

O objetivo não é apenas gerar imagens bonitas, mas gerar imagens:

- Rapidamente
- De forma interativa
- Dentro de restrições rígidas de tempo

Em geral, isso significa manter taxas como 30, 60 ou mais quadros por segundo.



Onde a CG em Tempo Real é usada?

Ela aparece em diversas áreas:

- Jogos digitais
- Simuladores (voo, direção, treinamento)
- Realidade virtual e aumentada
- Visualização científica e médica
- Interfaces gráficas ricas (HUDs, dashboards 3D)

Em todos esses casos, o sistema não pode parar para “renderizar”: a imagem precisa estar sempre pronta.

O desafio do Tempo Real

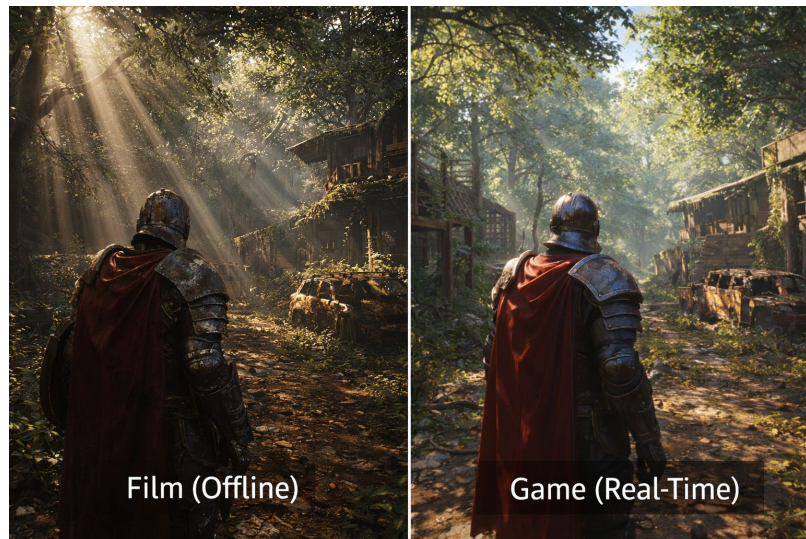
Em aplicações offline (cinema, animação), uma única imagem pode levar minutos ou horas para ser gerada.

Em tempo real:

- Cada frame tem apenas alguns milissegundos
- Todas as etapas gráficas precisam caber nesse tempo
- Decisões de desempenho impactam diretamente a qualidade visual

Isso força o uso de:

- Aproximações visuais
- Modelos simplificados
- Otimizações agressivas



Pipeline Gráfico: visão geral

O pipeline gráfico organiza o processo de transformar dados 3D em pixels na tela.

De forma simplificada, ele responde à pergunta:

Como um conjunto de vértices vira uma imagem?

O pipeline é composto por etapas sequenciais, cada uma com uma função clara:

- Processamento geométrico
- Transformações de coordenadas
- Cálculo de iluminação
- Conversão para pixels

Pipeline e GPU

Nas aplicações modernas, o pipeline gráfico é executado principalmente na **GPU**.

A GPU é especializada em:

- Processar milhares de vértices em paralelo
- Executar cálculos matemáticos simples em grande volume
- Rasterizar primitivas rapidamente

Por isso, o pipeline foi pensado para ser:

- Massivamente paralelo
- Altamente estruturado
- Otimizado para operações vetoriais

Etapas principais do pipeline

De forma conceitual, podemos dividir o pipeline em:

- Entrada de dados geométricos (vértices, malhas)
- Transformações espaciais (mundo, câmera, projeção)
- Definição do que é visível
- Cálculo de cor e iluminação
- Rasterização (pixels na tela)

Essas etapas não existem por acaso:

- cada uma resolve um problema específico da renderização.

Pipeline como fio condutor da disciplina

Todos os tópicos da disciplina se encaixam em algum ponto do pipeline:

- Modelagem geométrica → dados de entrada
- Culling e visibilidade → evitar trabalho desnecessário
- Iluminação, materiais e texturas → cálculo de cor
- Anti-aliasing e sombras → qualidade visual final

Entender o pipeline ajuda a entender **por que cada técnica existe e onde ela atua.**

Modelagem Geométrica em Computação Gráfica

Modelagem geométrica trata de como os objetos são descritos matematicamente para que possam ser processados pelo pipeline gráfico.

Esses modelos definem:

- Forma
- Tamanho
- Estrutura espacial

Antes de iluminação, textura ou sombra, tudo começa com a geometria.

Tipos de Representação Geométrica

Existem diferentes formas de representar um objeto no espaço 3D.

De maneira geral, podemos dividir em:

- **Representações explícitas:** descrevem diretamente a superfície
- **Representações implícitas:** descrevem o objeto por meio de equações

Em computação gráfica em tempo real, a escolha da representação impacta diretamente o desempenho.

Representação Implícita (ideia geral)

Na representação implícita, o objeto é definido por uma função matemática, como por exemplo:

- $f(x, y, z) = 0$

Esse tipo de representação é comum em:

- Metaballs
- Campos de distância
- Algumas simulações físicas

Apesar de poderosa, **não é a mais usada diretamente na GPU** para renderização em tempo real.

Representação Explícita: Malhas

Na prática, a representação dominante em tempo real é a **malha poligonal**.

Uma malha é composta por:

- Vértices (pontos no espaço)
- Arestas (ligações entre vértices)
- Faces (superfícies planas)

Essa estrutura é simples, direta e altamente compatível com o pipeline gráfico.

Por que usar Triângulos?

Embora existam polígonos com mais lados, **tudo acaba sendo convertido em triângulos.**

Motivos principais:

- Três pontos definem sempre um plano
- Triângulos nunca são ambíguos
- A GPU é otimizada para processar triângulos

Independentemente da ferramenta ou engine, o resultado final enviado para a GPU é uma **lista de triângulos.**

Triangularização

Triangularização é o processo de:

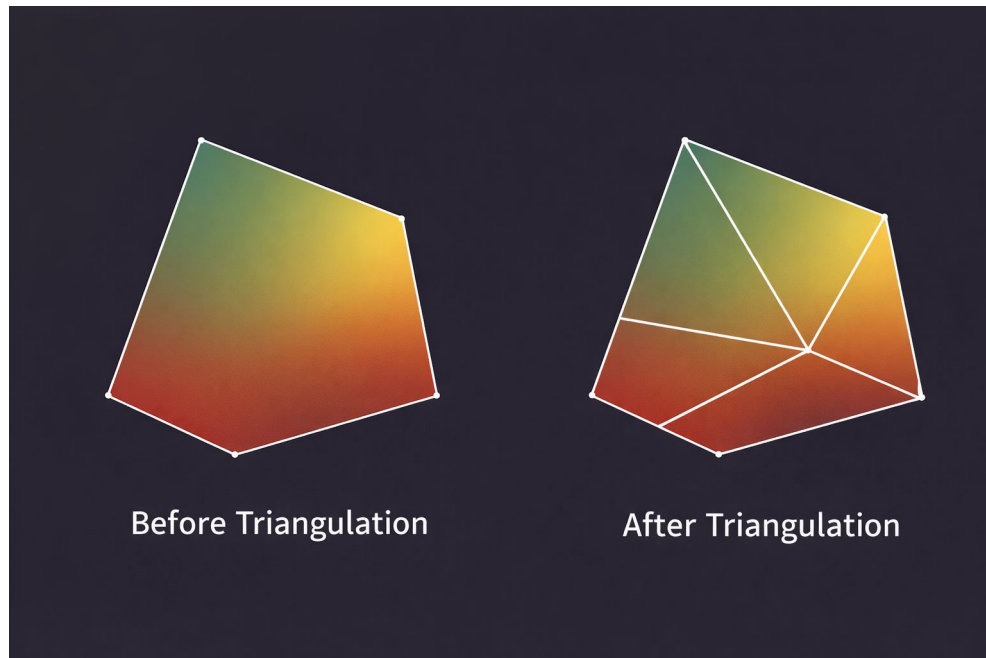
- Converter polígonos complexos
- Em conjuntos de triângulos equivalentes

Ela pode acontecer:

- No software de modelagem
- Na engine
- Automaticamente na GPU

Uma má triangularização pode gerar:

- Artefatos visuais
- Problemas de iluminação



Tessellation: mais detalhe sem mudar o modelo base

Tessellation subdivide triângulos em triângulos menores **durante a renderização**.

Isso permite:

- Aumentar detalhe visual
- Adaptar nível de detalhe à distância da câmera
- Manter modelos base simples

Muito usada para terrenos, superfícies orgânicas e efeitos de relevo.

Consolidação de Malhas

Consolidação busca **reduzir o custo de renderização** agrupando dados geométricos.

Ideias comuns:

- Unir malhas estáticas
- Reduzir número de chamadas de desenho
- Compartilhar vértices quando possível

Menos objetos enviados à GPU geralmente significa **mais desempenho**.

Simplificação Geométrica

Simplificação reduz a quantidade de triângulos mantendo a forma geral do objeto.

Usada principalmente para:

- Objetos distantes
- Sistemas de LOD (Level of Detail)
- Otimização em tempo real

O desafio é reduzir complexidade **sem perda visual perceptível**.

Geometria como base de tudo

A geometria define:

- O que pode ser visto
- Onde a luz incide
- Onde sombras são projetadas

Decisões geométricas impactam diretamente:

- Qualidade visual
- Desempenho
- Complexidade dos próximos estágios do pipeline

A partir daqui, o próximo problema natural é decidir **o que realmente precisa ser desenhado na cena** - visibilidade.

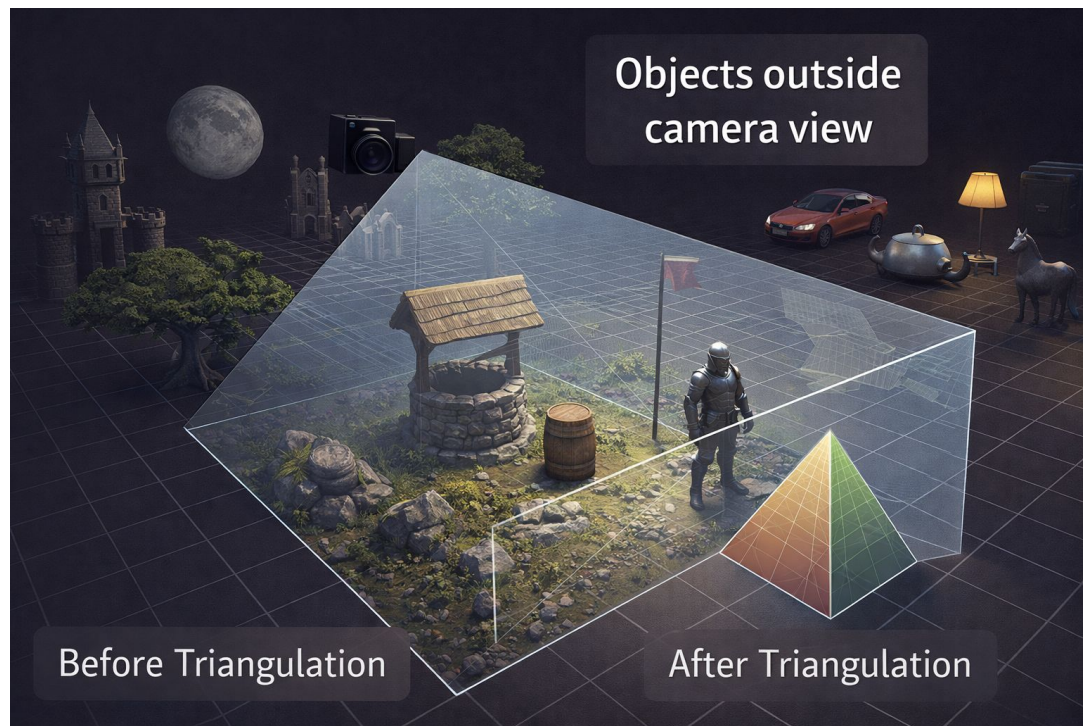
O problema da visibilidade

Em uma cena 3D, **nem tudo precisa ser desenhado.**

Mesmo objetos corretamente modelados e posicionados podem:

- Estar fora da câmera
- Estar voltados para longe
- Estar completamente escondidos por outros objetos

Processar tudo indiscriminadamente desperdiça tempo de GPU.



Visibilidade no pipeline gráfico

Visibilidade trata de decidir **o que realmente contribui para a imagem final**.

Ela atua antes ou durante a rasterização para:

- Reduzir quantidade de triângulos processados
- Evitar cálculos de iluminação desnecessários
- Melhorar desempenho em tempo real

É uma etapa crítica em aplicações interativas grandes, como jogos de mundo aberto.

Culling: descartando o que não importa

Culling é o nome dado às técnicas de **descartes geométricos automáticos**.

A ideia central é simples:

- Se algo não pode aparecer na imagem, não deve ser processado.

Existem diferentes tipos de *culling*, cada um resolvendo um problema específico.

Back-face Culling

Back-face culling remove faces que estão **voltadas para longe da câmera**.

Isso funciona porque:

- Objetos sólidos têm faces internas invisíveis
- Apenas a “frente” do polígono pode ser vista

Resultado:

- Aproximadamente metade das faces pode ser descartada
- Ganho significativo de desempenho com custo mínimo

Frustum Culling

Frustum culling remove objetos **fora do volume de visão da câmera**.

O frustum é definido por:

- Campo de visão
- Plano próximo
- Plano distante

Se um objeto está completamente fora desse volume, ele não pode aparecer na tela.



Frustum Culling remove objetos **fora do volume de visão** da câmera



Occlusion Culling

Occlusion culling descarta **objetos escondidos atrás de outros objetos**.

Mesmo estando dentro do frustum:

- Um objeto pode estar totalmente oculto
- Renderizá-lo não altera a imagem final

Essa técnica é mais complexa e geralmente:

- Usa testes de profundidade
- Depende de heurísticas
- Tem custo maior que back-face e frustum culling



Culling e desempenho em tempo real

As técnicas de culling:

- Não alteram o resultado visual correto
- Reduzem drasticamente o custo computacional
- São essenciais para escalar cenas complexas

Em engines modernas, essas técnicas são aplicadas automaticamente, mas **entender como funcionam** ajuda a:

- Modelar cenas melhores
- Evitar gargalos
- Tomar decisões de design gráfico

O que vem depois da visibilidade

Depois de decidir:

- Quais objetos são visíveis
- Quais faces realmente importam

O próximo passo do pipeline é determinar **como esses objetos vão parecer**, o que depende de:

- Luz
- Material
- Sombreamento

A partir daqui, o foco passa a ser **iluminação e modelos de sombreamento**.

Por que iluminação é necessária?

Sem iluminação, uma cena 3D é apenas um conjunto de formas sem profundidade visual.

A iluminação é responsável por:

- Dar sensação de volume
- Destacar relevo e curvatura
- Indicar posição espacial dos objetos
- Tornar a cena compreensível visualmente

Dois objetos com a mesma geometria podem parecer completamente diferentes apenas mudando a iluminação.



Iluminação em CG: uma aproximação

Em computação gráfica em tempo real, **não simulamos a física real da luz.**

Em vez disso, usamos:

- Modelos matemáticos simples
- Aproximações visuais
- Equações computacionalmente baratas

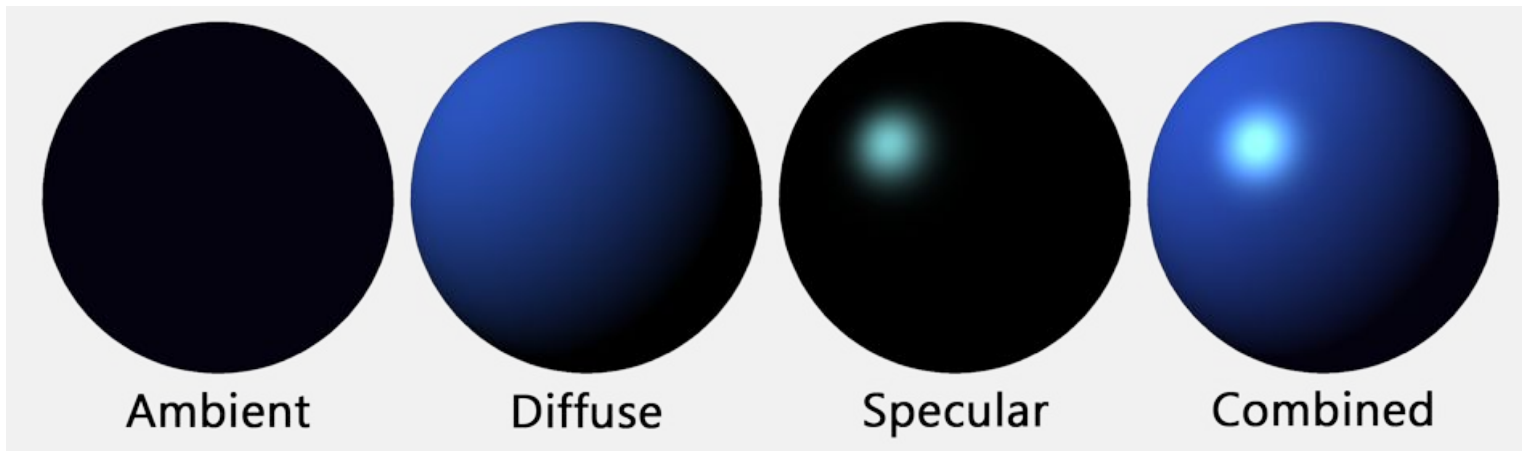
O objetivo é parecer realista, não ser fisicamente exato.

Componentes básicos da iluminação

A maioria dos modelos de iluminação locais combina três componentes:

- **Ambiente:** luz geral da cena
- **Difusa:** luz refletida igualmente em todas as direções
- **Especular:** brilho concentrado, dependente do observador

Esses componentes são combinados para calcular a cor final de cada ponto da superfície.



Representação de fontes de luz

Para calcular iluminação, a cena precisa de **fontes de luz artificiais**.

Cada luz possui propriedades como:

- Tipo
- Intensidade
- Cor
- Direção ou posição

A escolha do tipo de luz influencia diretamente o resultado visual e o custo computacional.

Luz Direcional

A luz direcional simula uma fonte muito distante, como o sol.

Características:

- Não possui posição definida
- Todos os raios são paralelos
- Iluminação uniforme na cena

É barata computacionalmente e muito usada para iluminação principal.

Luz Pontual

A luz pontual emite luz a partir de um ponto no espaço.

Características:

- Possui posição
- Emite luz em todas as direções
- Intensidade geralmente decai com a distância

Exemplos: lâmpadas, tochas, pequenas fontes de luz.

Luz Spot

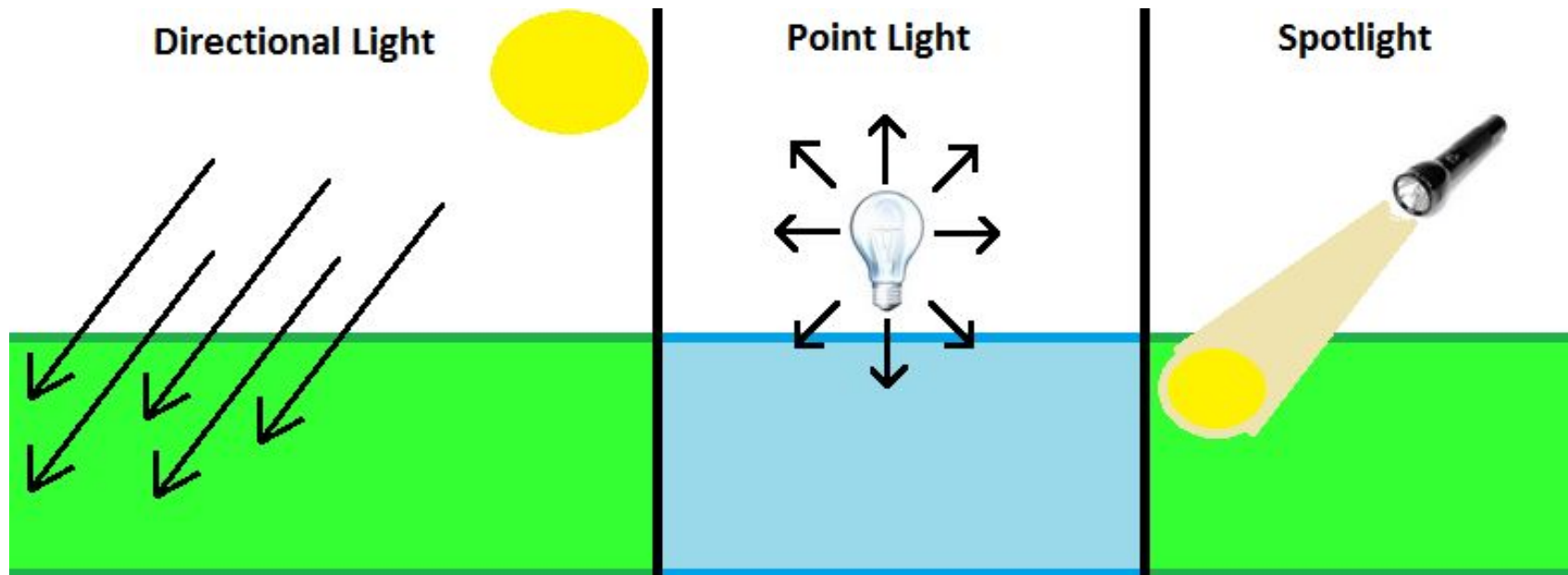
A luz spot combina posição e direção.

Características:

- Emite luz dentro de um cone
- Possui ângulo de abertura
- Ideal para efeitos direcionais

Exemplos: lanterna, holofote, farol.

Comparação



Modelos de Sombreamento

O modelo de sombreamento define **como a iluminação é calculada na superfície**.

Ele determina:

- Onde o cálculo acontece
- Como a cor varia na superfície
- O custo computacional do processo

Modelos diferentes produzem resultados visuais distintos, mesmo com a mesma luz e geometria.

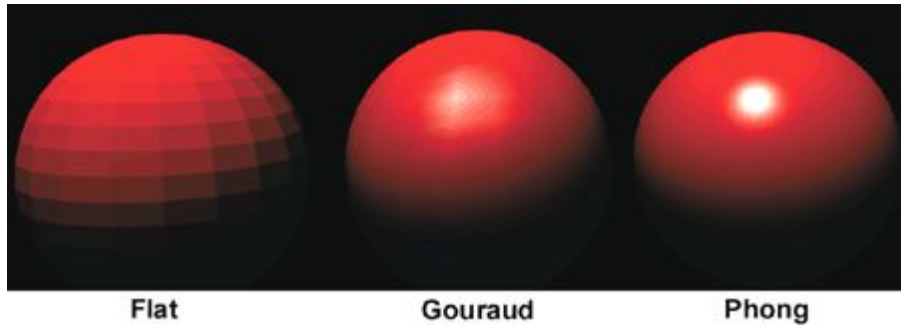
Flat, Gouraud e Phong (visão geral)

Três modelos clássicos:

- **Flat:** uma cor por face
- **Gouraud:** iluminação calculada nos vértices
- **Phong:** iluminação calculada por fragmento

Eles representam um trade-off entre:

- Qualidade visual
- Custo computacional



Iluminação Local: Modelo de Phong

O modelo de Phong é um dos mais usados em CG clássica.

Ele combina:

- Componente ambiente
- Reflexão difusa (Lei de Lambert)
- Reflexão especular (dependente do observador)

O cálculo é feito **por fragmento**, resultando em:

- Brilhos mais suaves
- Melhor percepção de curvatura

O papel do material na iluminação

A iluminação sozinha não define a aparência final.

Cada objeto possui um **material**, que controla:

- Quanto reflete luz
- Quão brilhante é o especular
- Como responde à iluminação

Dois objetos sob a mesma luz podem parecer:

- Plástico
- Metal
- Borracha
- Cerâmica



Iluminação como base da aparência

A iluminação define:

- Volume
- Profundidade
- Realismo visual

A partir dela, entram técnicas que **refinam a aparência**, como:

- Texturas
- Mapas auxiliares
- Reflexões do ambiente

O próximo passo do pipeline é enriquecer a superfície sem aumentar a geometria - texturização e materiais.

Por que usar texturas?



Aumentar o número de triângulos melhora o detalhe, mas custa caro.

Texturas permitem:

- Adicionar detalhes visuais finos
- Simular complexidade geométrica
- Manter desempenho em tempo real

Na prática, grande parte do “realismo” vem de **texturas**, não da geometria.

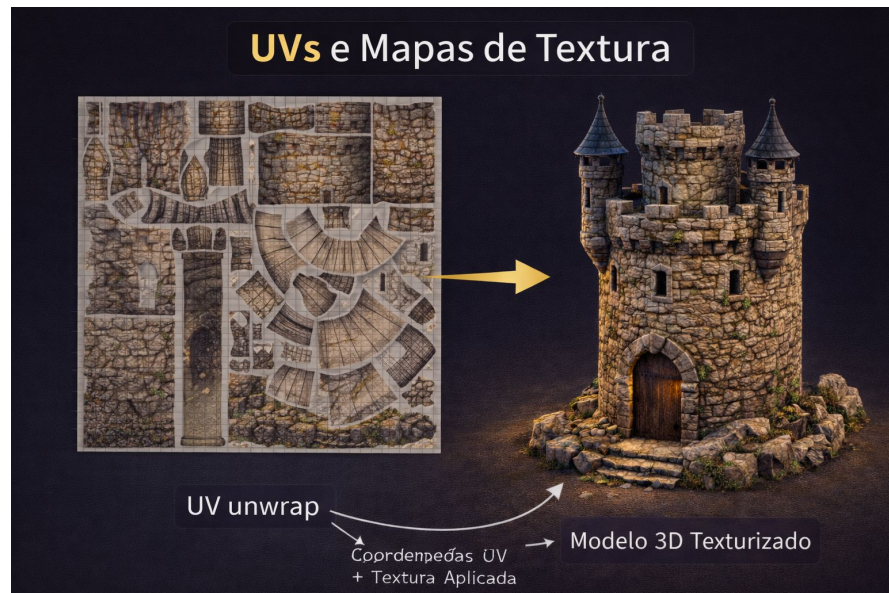
O que é uma textura?

Uma textura é uma **imagem aplicada sobre a superfície** de um objeto 3D.

Ela define propriedades por ponto da superfície, como:

- Cor
- Brilho
- Rugosidade
- Direção da normal

A aplicação da textura usa coordenadas UV, que mapeiam a imagem 2D na superfície 3D.



Textura difusa (albedo)

A textura difusa define a **cor base do material**, sem efeitos de iluminação.

Ela representa:

- Pigmento do objeto
- Padrões visuais (madeira, pedra, tecido)

Importante: ela não **contém sombra nem brilho**, apenas cor.

Normal Map: detalhe sem geometria

Normal maps alteram a **direção da normal da superfície**, sem mudar a malha.

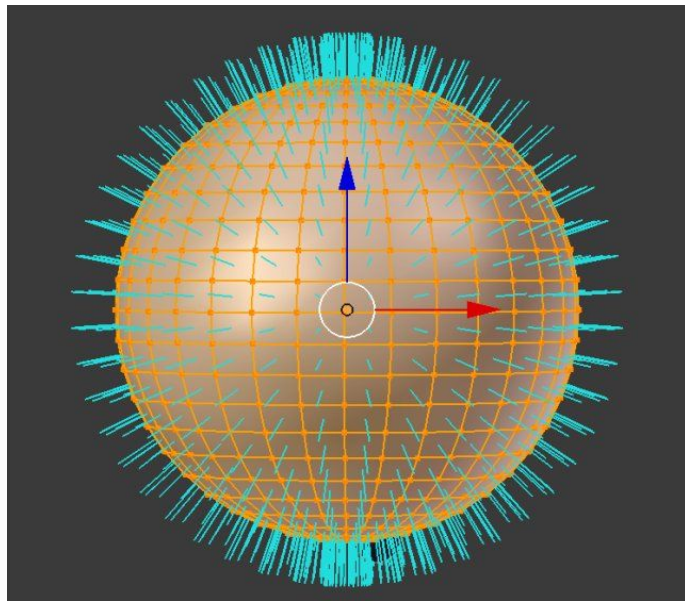
Isso permite:

Simular relevos

Criar sensação de profundidade

Manter o modelo leve

Visualmente, o objeto parece mais detalhado, mas continua simples geometricamente.



Mapas especulares e de rugosidade

Esses mapas controlam como a luz reflete no material.

Exemplos:

- **Specular map:** onde o brilho aparece
- **Roughness map:** quão espalhado é o brilho

Eles ajudam a diferenciar materiais como:

- Metal polido
- Plástico fosco
- Borracha

Material: combinando tudo

Um material é a **combinação de vários parâmetros**, como:

- Modelo de iluminação
- Texturas associadas
- Propriedades ópticas

O material define como o objeto:

- Reage à luz
- Reflete o ambiente
- Se destaca visualmente na cena

Na engine, material é uma entidade lógica que conecta shaders e dados.

Environment Mapping: reflexões do ambiente

Environment mapping simula reflexões do ambiente sem traçar raios reais.

A ideia é:

- Usar uma imagem do ambiente
- Projetá-la como reflexão no objeto

Muito usado para:

- Metais
- Superfícies brilhantes
- Água e vidro simplificados



Cube Maps e Skyboxes

Uma forma comum de environment mapping é o **cube map**.

Ele representa o ambiente como:

- Seis imagens formando um cubo
- Uma visão em todas as direções

Além de reflexões, cube maps também são usados como:

- Skybox
- Fundo da cena

Aparência sem custo geométrico

Texturas e materiais permitem:

- Alta qualidade visual
- Baixo custo de processamento
- Grande variedade de estilos visuais

Depois de definir **como o objeto parece**, surgem problemas ligados à **qualidade da imagem gerada**, como serrilhado e sombras incorretas.

O próximo conjunto de técnicas trata exatamente disso: **aliasing, anti-aliasing e sombras**.

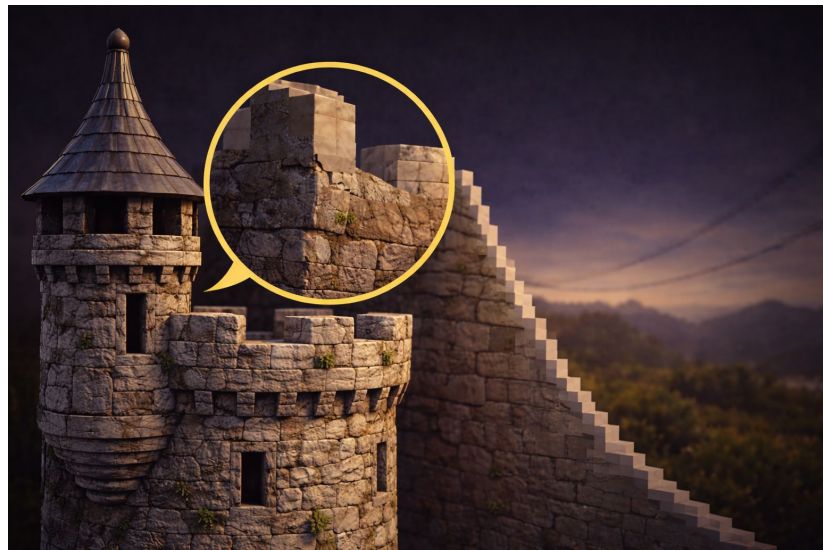
Aliasing: o problema das bordas serrilhadas

Aliasing é um artefato visual causado pela conversão de formas contínuas em pixels discretos.

Ele aparece principalmente em:

- Bordas inclinadas
- Linhas finas
- Objetos distantes

O efeito é conhecido visualmente como “**escadinhas**”.



Por que o aliasing acontece?

A tela é uma grade fixa de pixels.

Quando uma borda não se alinha exatamente com essa grade, ocorre perda de informação.

Quanto menor a resolução:

- Mais perceptível o aliasing
- Maior o impacto visual

Esse é um problema **estrutural da rasterização**, não um erro de implementação.

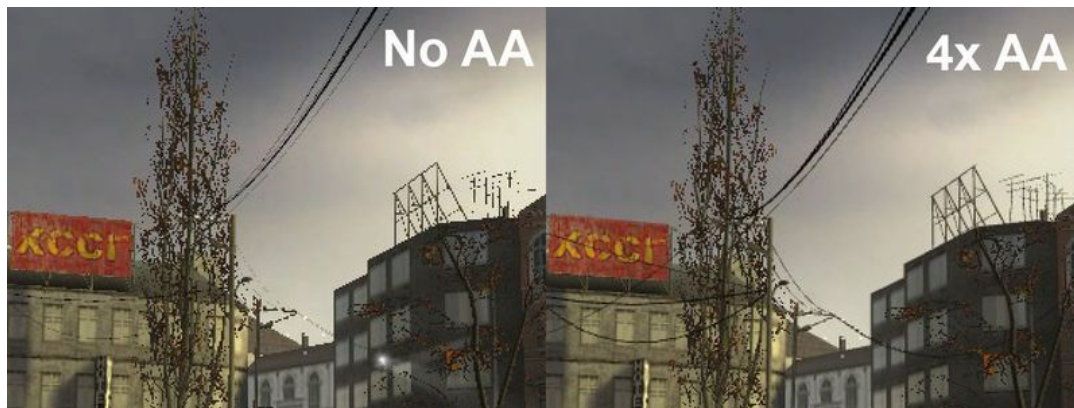
Anti-aliasing: suavizando a imagem

Anti-aliasing é um conjunto de técnicas para **reduzir o serrilhado**.

A ideia geral é:

- Amostrar mais informações
- Suavizar transições de cor
- Reduzir contrastes abruptos nas bordas

O resultado é uma imagem visualmente mais suave.



Tipos comuns de Anti-aliasing (visão geral)

Algumas abordagens frequentes:

- **MSAA**: múltiplas amostras por pixel
- **FXAA**: pós-processamento baseado em bordas
- **TAA**: acumulação temporal de frames

Cada técnica envolve trade-offs entre:

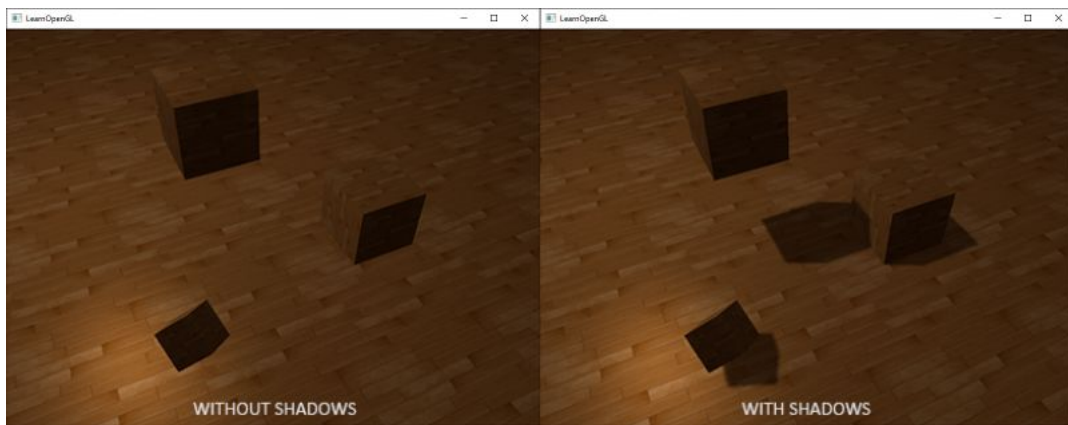
- Qualidade
- Custo computacional
- Estabilidade da imagem

Sombras: por que são importantes?

Sombras ajudam o observador a perceber:

- Profundidade
- Distância entre objetos
- Contato com o chão

Sem sombras, a cena tende a parecer “flutuante” e artificial.



O desafio das sombras em tempo real

Calcular sombras fisicamente corretas é caro.

Em tempo real, usamos:

- Aproximações
- Projeções
- Técnicas baseadas em mapas

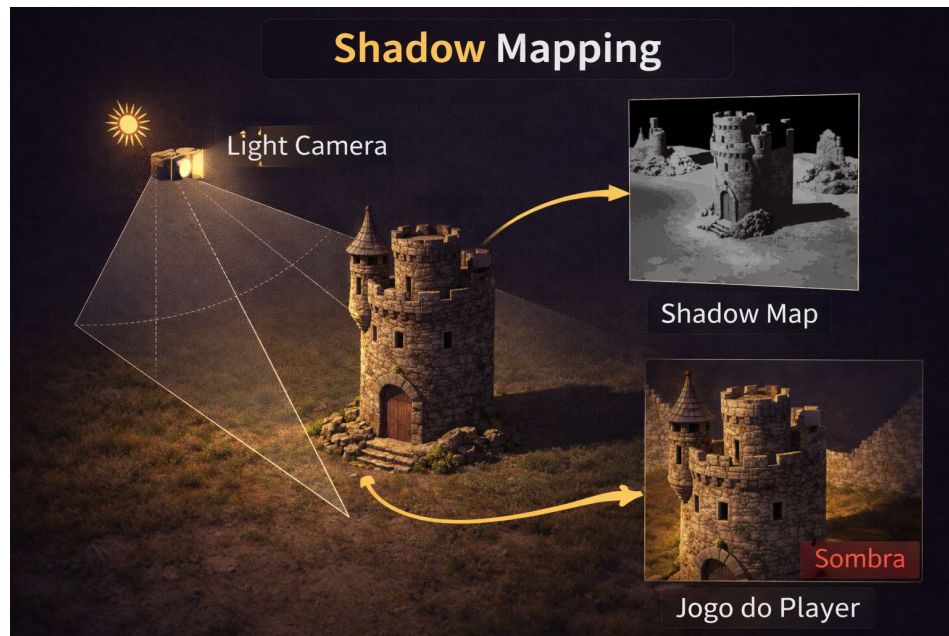
O objetivo é gerar sombras **convincentes**, não perfeitamente corretas.

Shadow Mapping (ideia central)

Shadow mapping é a técnica mais comum em tempo real.

Funcionamento geral:

- A cena é renderizada do ponto de vista da luz
- Gera-se um mapa de profundidade
- Durante a renderização final, testa-se se o ponto está na sombra



Limitações do Shadow Mapping

Problemas comuns:

- Bordas serrilhadas
- Acne de sombra
- Vazamento de luz

Soluções envolvem:

- Ajustes de bias
- Filtros de suavização
- Resoluções maiores de mapa

Esses problemas reforçam a ideia de **trade-off entre qualidade e desempenho**.

Renderização de texto em CG

Texto em aplicações gráficas não é “desenhado”, é **renderizado**.

Abordagens comuns:

- Texturas com atlas de caracteres
- Fontes rasterizadas
- Fontes vetoriais convertidas em malha

Muito usado em:

- HUDs
- Interfaces
- Informações em tempo real

Texto como elemento gráfico

Mesmo texto passa pelo pipeline gráfico:

- Possui geometria (quads)
- Usa texturas
- Pode sofrer iluminação ou pós-processamento

Isso reforça que, em CG, tudo vira **geometria + material + rasterização**.

Conectando todos os conceitos

Ao longo da aula, vimos como:

- A geometria define a base da cena
- A visibilidade evita trabalho desnecessário
- A iluminação e os materiais definem aparência
- Anti-aliasing e sombras refinam o resultado
- Texto e interfaces também seguem o pipeline

Todos esses elementos trabalham juntos para gerar uma **imagem interativa, contínua e eficiente em tempo real**.

Técnicas Poligonais como controle de complexidade

Em computação gráfica em tempo real, o problema central não é apenas **desenhar**, mas **desenhar dentro do tempo disponível**.

As técnicas poligonais existem para controlar:

- Quantidade de triângulos
- Custo de processamento na GPU
- Qualidade visual percebida

Elas atuam diretamente sobre a **entrada geométrica do pipeline**.

Triângulos, detalhe e custo

Quanto mais triângulos:

- Maior o detalhamento geométrico
- Maior o custo de transformação, iluminação e rasterização

Por isso, o detalhe geométrico **não é uniforme**:

- Objetos próximos podem ser mais detalhados
- Objetos distantes podem ser simplificados
- Superfícies podem ganhar detalhe via tessellation apenas quando necessário

Essa variação dinâmica é essencial em aplicações em tempo real.

Todas as técnicas trabalhando juntas

Na prática, uma cena em tempo real combina:

- Malhas triangulares bem definidas
- Simplificação e LOD para distância
- Consolidação para reduzir overhead
- Tessellation para detalhe localizado
- Culling para eliminar trabalho inútil

Nada disso é isolado.

Cada decisão geométrica afeta:

- Visibilidade
- Iluminação
- Sombras
- Qualidade final da imagem